

Menadevs AI Engineer

Duration: 48 Hours **Deliverables:** GitHub Repository (Private) + 5-min Video Walkthrough

Provided Files

You should have the following files to start:

- `README.md`: This instruction file.
- `setup_data.py`: Script to generate the SQL database and PDF links.
- `golden_dataset.json`: The ground truth QA pairs for evaluation.
- `evaluation_harness.py` (Optional): A skeleton script to help structure your grading.

1. The Scenario

You are the lead architect for a financial intelligence platform. Your users—investment analysts—need to ask complex questions that require synthesizing information from two distinct sources:

1. **Unstructured Knowledge:** 10-K Annual Reports (PDFs) describing risks, strategy, and legal proceedings. *Note: These documents contain complex tables.*
2. **Structured Data:** A SQL database containing historical daily stock prices.

Your Goal: Build a production-ready **Hybrid RAG Agent** that intelligently routes queries, generates SQL, retrieves text/tables, and synthesizes a final answer.

2. The Setup (Getting Started)

Step A: Generate the Environment

We have provided a script `setup_data.py`. Run it to generate the SQL database (`finance.db`) and get the links to the required PDF documents.

```
python setup_data.py
```

Note: The script uses a fixed seed (42). The generated data will be consistent for all candidates.

Step B: The Stack Requirements

You are free to use any libraries, but your solution **must** adhere to these constraints:

- **Model:** Use an open-source model (e.g., `Llama-3-8b`, `Mistral-7b`). **Do not use OpenAI/Anthropic APIs.**
- **Orchestration:** Use a graph-based or agentic framework (e.g., LangGraph, AutoGen, or a custom State Machine).
- **Database:** Use the generated SQLite DB and a local vector store (ChromaDB, FAISS, or Qdrant).

Hardware Reality Check

We understand not everyone has an H100 GPU at home.

- **No GPU?** Use `Ollama` or `llama.cpp` to run 4-bit quantized models on CPU/Mac Metal.
- **Grading Focus:** We care about the **architectural pattern** (how you handle routing, caching, and state), not the raw inference speed of your laptop.

3. The Challenge Requirements

Part I: The Reasoning Engine (Agentic Workflow)

Build a Python application (using LangGraph, AutoGen, or raw Python) that serves an API endpoint `POST /query`.

1. Intelligent Routing: The system must classify if a user prompt requires:

- **Retrieval** (looking up text/tables in PDFs).
- **SQL Execution** (querying the database).
- **Both** (e.g., "Compare the 2023 risk factors of Apple with its stock volatility in Q4.").

2. Tool Use (SQL Generation): The LLM must generate the SQL query dynamically based on the user's natural language request. It cannot have pre-canned SQL queries.

3. Advanced Retrieval (The "Table Trap"): Standard RAG fails on financial tables. Your ingestion pipeline must be robust.

- **Requirement:** The PDFs contain dense tables (e.g., "Revenue by Product Category"). Naive text chunking will fail. You must use a strategy for **Table Parsing** (e.g., `unstructured`, `gmft`, `pdfplumber`, or Vision Models) to accurately retrieve this data.

Part II: The "Level 2" Engineering (Optimization)

A Senior Engineer cares about inference costs, latency, and system architecture.

- **Inference Engine:** Serve the model using **vLLM** or **Ollama** to demonstrate handling of KV-cache and throughput.
- **Quantization:** The model must run in **4-bit quantization** (GGUF or AWQ) to demonstrate efficient memory management.
- **Hybrid Search:** Your retrieval pipeline must implement **Hybrid Search** (Keyword/BM25 + Semantic Vector) + **Re-ranking**. Pure vector search is insufficient for specific financial terms.
- **Latency Optimization:** The API must support **Server-Sent Events (SSE)** (Streaming) so the user sees the answer token-by-token. Ideally, implement **Semantic Caching** so repeated queries do not hit the LLM.

Part III: Evaluation (The Quality Gate)

An unmeasured system is a broken system. Prove your system works.

1. **The "Judge":** Implement an evaluation script using "LLM-as-a-Judge" (e.g., using a

stronger model or a separate prompt) to grade your own answers. *You may use the provided `evaluation_harness.py` as a starting point.*

2. **Metrics:** Run the provided `golden_dataset.json` (10 questions) and output a `eval_results.csv` containing scores for:
 - **Context Precision:** Did the RAG retrieval actually find the relevant chunk?
 - **Faithfulness:** Did the final answer hallucinate info not present in the source?
 - **SQL Validity:** Did the generated SQL query execute without syntax errors?
3. **CI/CD:** Include a script (e.g., `run_ci.sh` or GitHub Actions) that automatically runs these evals.

Part IV: Bonus "Boss Level" Challenge (Optional)

Modify the system to handle **Arabic queries** (e.g., 'ما هي أرباح شركة آبل؟').

- The system should translate the query to English for processing.
- Retrieve the data/answer.
- Translate the final response back to Arabic.

4. The Grading Rubric

Here is exactly how we will assess your code. We are looking for "Level 2" engineering practices.

Feature	● Fail (Junior Mistake)	● Pass (Senior Implementation)
Ingestion (Chunking)	Naive splitting (breaks tables/sentences).	Semantic Chunking / Table Parsing (Preserves structure).
SQL Generation	Blindly executes LLM-generated SQL (SQL Injection risk).	Implements a Schema Check or restricted scope (read-only) before execution.
Retrieval	Simple Cosine Similarity only.	Hybrid Search (Keyword + Vector) + Re-ranking (Cross-Encoder).
Architecture	A massive <code>if/else</code> chain in <code>main.py</code> .	Modular design : Separated <code>ingest.py</code> , <code>agent.py</code> , and <code>eval.py</code> .
Latency/UX	Waits 10s then dumps text.	Streaming API (SSE) + Caching.
Docker	"It runs on my machine."	Includes a <code>Dockerfile</code> that sets up the environment and model weights automatically.

5. Submission Guidelines

Anti-Cheating Policy: Plagiarism or using public code from other candidates will result in immediate disqualification.

Please follow these steps to submit your work:

1. **Create a PRIVATE GitHub Repository:** Do **not** create a public repository.
2. **Invite Collaborators:** Invite `[Insert_Hiring_Manager_GitHub_Handle]` as a collaborator to your private repo so we can review the code.
3. **Repository Contents:**
 - **Source Code:** Modular, typed Python code.
 - **eval_results.csv:** The output of your evaluation run against the 10 golden questions.
4. **Video Walkthrough:** Include a link (Loom/YouTube Unlisted) to a < 5-minute video. Walk us through a complex query trace (e.g., showing the agent "thinking", calling the SQL tool, and retrieving text). **We verify authorship primarily through this video explanation.**

Good luck!